

SECAUDIT 054 - desktop suite <-> CMS attack surface: paranoid duty-of-care pass (image-decoder RCE, config poisoning, webview->desktop code-exec, credentials, server parser)

SECURITY AUDIT / PUBLIC REMEDIATION RECORD

Field	Value
Audit ID	2026-09-02-054
Date	2026-09-02
Severity	HIGH - no confirmed live RCE, but multiple HIGH paths to code execution or full-site credential compromise across the desktop<->CMS boundary that had never had a coordinated adversarial pass. One CRITICAL config-poisoning path (credential exfiltration via a tampered profile). The server ingest side is strong; the desktop side is where the exposure concentrates.
Scope	The whole desktop<->CMS boundary and the desktop suite's untrusted-input surfaces, per <code>_spec/SPEC-desktop-security-duty-of-care-v0_2.docx</code> (6 chokepoints) + <code>_spec/SPEC-desktop-cms-mutual-auth-v0_1.docx</code> . Files across <code>tools/hub/</code> (SNAP SLAPPER), <code>tools/gyss/</code> , <code>tools/coldsnap/</code> , <code>tools/_shared/</code> , <code>tools/smack-up-your-backup/</code> , and <code>core/</code> server ingest.
Method	Four independent adversarial reviewers (one per chokepoint cluster) + author pass, each grounded in live code with file:line, rated by FAIL-CLOSED enforcement (not "today's caller is safe"). No exploit code. Per the 050 rule: never collapse to a single self-audit.
Status	CLOSED (code) 2026-09-18 - every code item in §8 is fixed on dev (most landed between Sep 2 and Sep 13 and were never recorded here; the last three landed in 0.7.724D). Three items are not code and stay OPEN as decisions/process: mandatory credential vault, server-side scope enforcement + config signing (both three-way with Sean), and the duty-of-care process items. See §10. Original working status for the record: Fixed this pass: GYSS data-id XSS sink (esc), and store_media ext-jail (carried from 053). OPEN + prioritised below. The GYSS CSP (the top code-exec control) is confirmed still null and needs a build+live-test to close. Several items are server-side / architecture -> three-way sign-off.
Reporter	Sean (commissioned a maximally-paranoid pass: "be as fucking paranoid as you can... think of bizarre edge cases") + Claude (4 reviewers + synthesis).
Related	Supersedes the narrow 053 (branch-diff review - see section 6). Confirms/advances 039 (GYSS webview: Finding A closed, Finding F/CSP still open). Extends 044 (shared profiles), 049 (post model), 050 (multi-reviewer discipline). Companion to the mutual-auth spec (server-side A1). Numbering note: 052 = the llms.txt agent-surface audit; the COLD SNAP branch review is 053 ; this is 054 - the two-052 collision the mutual-auth spec flagged is hereby disambiguated.
Disclosure	Single-operator desktop tools + per-site CMS installs. Most desktop findings require the operator to connect a tool to a hostile/compromised server, open an attacker-authored project/texture, or a local disk write. The server ingest surface is unauthenticated-reachable via federation but verified strong. No exploitation known.

1. Summary

Sean's question, plainly: *can a hole in one half be used to screw the other, in both directions - a CMS-delivered miner/ransomware on a desktop, or a crafted image that runs code in SNAP SLAPPER?* The honest answer is that the **server side is hardened and the desktop side had never had the adversarial pass its surface demands** - which is exactly the asymmetry the duty-of-care brief names.

Good news first, because it bounds the fear: the **server ingest is the strongest code in the set**. Image uploads are re-encoded through GD (which strips polyglot tails), an execution-guard is written into the upload dir, HTML is run through a real DOM allowlist, SVG is sanitised, skin packages are Ed25519-signed, and **there is no server-side SSRF** in any ingest path. A crafted image *uploaded* to the CMS cannot become a stored payload.

The exposure is on the **desktop**, at a handful of chokepoints, and it is real:

- **A crafted image CAN reach a native decoder without the user picking it** - FOUND TEXTURES auto-downloads and decodes server-supplied bytes, and .slapper projects reference arbitrary paths/URLs - and nothing sets a Pillow pixel ceiling or a format allowlist. That is the "nasty in an image file" path, and it is HIGH.
- **The cleanest CMS->desktop code-execution chain is GYSS's null CSP** feeding a privileged file-write that can overwrite a tool exe the Hub later launches. That is the miner/ransomware delivery vector, and its key control (CSP) is still off.
- **A tampered profile silently sends a real posting key to an attacker** (config-as-action) - CRITICAL, and the mirror-image of the direction people under-weight.

Six chokepoints reviewed; findings ranked below by **how close they get an attacker to code execution or credential compromise**, per the brief's ranking rule.

2. Method - what was examined and how

Four reviewers ran independently and in parallel, each owning one chokepoint cluster, each told to try to break it and to rate by fail-closed enforcement at the boundary (the `store_media-hardened-even-though-the-caller-was-safe` standard), grounding every conclusion - safe or unsafe - in file:line:

1. **Chokepoint 1 - untrusted image ingress** (`tools/hub/ SNAP SLAPPER` + all Pillow/Qt decode paths, the RawTherapee/XPANO handoffs).
2. **Chokepoints 2 + 5 - config/profile/manifest ingestion + credential store at rest** (`tools/_shared/ snap_profiles/creds/vault/home/paths, snap_discovery`, the tool connection sinks).
3. **Chokepoints 3 + 4 - JS/IPC into the GYSS webview + update/restore channel** (Tauri `conf/capabilities/lib.rs`, `main.js` sinks, SUYB restore, archive extraction).
4. **Chokepoint 6 + AI trust + exotic** (server ingest SSRF, image-ingest hardening, AI model-output trust, XXE, polyglots, shared-state contamination, pickle/yaml/eval sweep).

No exploit code was produced; every item is a defensive control target. This four-reviewer structure is the process fix (section 7) and it earned its keep this pass - it surfaced the config-poisoning CRITICAL and re-confirmed the still-open CSP that the prior narrow pass had glossed.

3. Findings - ranked by reach to code execution / credential compromise

[CRITICAL] Profile `site_url` is acted on with zero validation -> a tampered profile exfiltrates a live posting key (config-as-action; Direction B)

`tools/coldsnap/sumna_post.py:145` builds `base_url` straight from the profile, attaches `Authorization: Bearer {api_key}`, and POSTs to it; source `snap_profiles.py:95-103` accepts any `site_url` with no scheme check, host allowlist, signature, or cross-check against the filename `site_key`. One write to `shared_library/profiles/<site>.json` flipping `site_url` to an attacker (keeping `api_key_enc`) sends that site's real posting key to the attacker, who then uses it against the real site. Hits every tool that reads the shared store.

Fail-closed fix: before building any authenticated session, run the existing `snap_stepup.insecure_transport_reason(url)` (today only the admin-password path uses it) and refuse; reject a profile whose in-file `site_url` host != its filename `site_key`. There is no signature on server config - document that residual (mutual-auth B1).

[HIGH] GYSS null CSP + privileged file-write -> overwrite a tool exe the Hub launches (the miner/ransomware chain)

tools/gyss/src-tauri/tauri.conf.json:24 "csp": null (039 Finding F, still open) + withGlobalTauri:true (:10) + app-defined write_file/download_to reachable from any webview script (lib.rs:49-58, ungated by capabilities, as the code comment states). The fs jail (resolve_in_root, 039 Finding A) IS remediated and confines writes to C:\snapsmack - but that root holds the PyInstaller tool exes, and the Hub launcher runs exact roster paths (044 only refuses wildcard globs, not overwrite of a known exe). Chain: hostile/compromised gyss-api.php field (or plain-http MITM) -> innerHTML -> script -> download_to attacker bytes over a roster .exe -> Hub launches it later. CSP is the one control that breaks the script-execution step regardless of any escaping bug. **Fail-closed fixes:** (a) set a real CSP - default-src 'self'; script-src 'self' <importmap-hash>; style-src 'self' 'unsafe-inline'; img-src 'self' https: asset: data:; connect-src 'self' https:; (b) refuse write_file/download_to whose destination is an existing executable or an exe-bearing dir (whitelist shared_library/**, config_files/gyss/**); (c) Hub verifies a known hash/signature of a roster exe before Popen. **Gate:** the inline importmap (index.html:10) and the global-dependent shim (tauri-core.js:5 uses window.__TAURI__) mean CSP and withGlobalTauri:false require a GYSS **rebuild + live all-four-tabs test** before they can be trusted (039's exact sequence; the tauri CLI is not installed on the audit box). Do NOT ship blind.

[HIGH] SNAP SLAPPER decodes untrusted image bytes with no pixel ceiling or format allowlist

Two genuinely-untrusted routes, not "own picks": FOUND TEXTURES auto-downloads server-supplied bytes and decodes them (textures_dialog.py:51 QImage.fromData on fetch_bytes, fired per search result), and .slapper projects reference arbitrary layer paths + restore_url (editor_window.py:2017-2021, editor_engine.py:150, 1639, 1646). Fleet-wide, Image.MAX_IMAGE_PIXELS/LOAD_TRUNCATED_IMAGES/format-allowlist are **never set** (0 hits across tools/). A crafted WebP/TIFF/ICO (the libwebp-zero-click class) or a decompression bomb has an open lane into Pillow and Qt's separate decoder stack. **Fail-closed fix (single highest-leverage control):** one import-early shared module - snap_imgsafe - that pins MAX_IMAGE_PIXELS to a real ceiling, keeps truncated-loading off, and exposes safe_open(path) that verifies format against an allowlist before .load(); route the FOUND TEXTURES cache decode, the .slapper layer decode, and the QImage.fromData thumbnail path through it (cap bytes + QImageReader.setAllowedFormats for the Qt path). This is a fleet-wide close at one chokepoint (COLD SNAP, GYSS thumbs, flkr-fckr benefit too). **Lane note: SNAP SLAPPER is Codex's active area - coordinate the in-file edits; the shared module is additive/in-lane.**

[HIGH] API keys / smackpress keys / AI + Drive secrets are base64-at-rest by default, not encrypted

snap_profiles.py:61-69 and coldsnap/profile_manager.py:62-70 are pure base64; snap_creds.py:66-70 falls back to b64: whenever the vault is locked, and nothing forces unlock (os.chmod 0o600 is a no-op on Windows). The vault (snap_vault.py, scrypt N=32768 + Fernet) is sound but **optional**. Anyone with disk read - malware, a synced/stolen laptop, a backup - recovers every key. Known lineage (036/037/039/050). **Fail-closed target:** the vault or OS keychain becomes the ONLY at-rest form for secrets; refuse to persist a secret as base64 on an at-rest-exposed machine. Owner/UX decision (passphrase-on-launch or keychain) - flagged for the beta bar.

[HIGH] Discovery trusts the hub node list wholesale; the hub's full key is POSTed to unvalidated spoke URLs

snap_discovery.py:148-180 POSTs Bearer {api_key_local} to each site_url taken from the server's multisite.nodes, and the insecure-transport guard covers only the admin-password branch. A compromised/MITM'd hub injects a node -> the hub's provisioning-capable key goes to an attacker URL, and the poisoned profile is written into the shared store every tool reads. **Fail-closed fix:** validate every node site_url with insecure_transport_reason; require the hub response over https on the Bearer branch; constrain spoke hosts to the hub's registrable domain / an allowlist.

[MEDIUM] External RAW/pano decoders run with no isolation, timeout, or network containment

`raw_handoff.py`:31-34 (fire-and-forget Popen) and `panomerge.py`:261-275 (QProcess). **Argument hygiene is already correct** (`argv` lists, `shell=False`, `abspath` so a filename can't pose as a flag, no attacker-supplied `.pp3/project` - verified safe, section 4) - so the *instruction* layer is closed. The residual is the *parser* layer: a RawTherapee/XPANO/LibRaw parser RCE runs unconfined with network + the user's token. **Fail-closed fix (containment, since we can't fix their parsers)**: launch through a Windows Job Object (kill-on-close, memory cap), add a wall-clock timeout, treat their output dir as untrusted, drop network where feasible. This is the answer to "we can't secure RawTherapee/XPANO but can we contain them" - yes: instruction closed, parser contained.

[MEDIUM] Other desktop + server items

- **GYSS download_to SSRF primitive** (`lib.rs`:160-189): `http(s)` + jailed dest, but not CORS-bound and follows redirects; a malicious manifest URL reaches intranet/loopback. Fix: pin to the connected site's origin, disable redirects, block private/loopback ranges.
- **Backup restore not authenticity-verified** (`restore_engine.py`:58-107, 213-228): integrity is checked against a manifest *inside* the same archive (defends corruption, not substitution). A replaced cloud/local backup writes attacker content (incl. `live.php`) onto the site during restore; Zip-Slip itself is contained. Fix: sign archives at creation, verify desktop-side before restore (model on `_build/sign-release.php`).
- **RSS fetch DNS-rebind TOCTOU** (`cron-rss-fetch.php`:46-69, 126): validates the host then fetches by hostname again; the sibling `cron-directory-feeds.php` does it right with `CURLOPT_RESOLVE` pinning. Fix: mirror the sibling. (Admin/hub-set host -> limited exposure.)
- **AI title/caption stored semi-raw** (`gyss-api.php`:653-660): only `alt` is sanitised; title/caption get a length cap only. A crafted image or a fleet-synced poisoned prompt can seed markup/shortcodes that execute wherever a skin echoes `img_description` unescaped. Fix: sanitise title/caption at the storage boundary the way `alt` already is (model output is DATA).
- **Untrusted SVG layer** parsed by QtSvg with no gate (`editor_engine.py`:149-174); **restore-URL** in a `.slapper` project drives an app-authenticated fetch; **configured external-exe path** trusted from prefs without signature check. Fixes: gate/size-cap SVG, origin-pin restore fetches, validate the exe.

[MEDIUM/LOW] Transport verification defaults off

FTPS `verify_cert` and SFTP host-key verification default to permissive (`transport.py`:38, 53), so an active MITM on the file-transfer channel can present any cert. The higher-value admin-login path is separately hard-refused over plain http (regression-tested). Fix: default verification ON with a deliberate per-profile opt-out (the `suyb_known_hosts` pin file already exists).

4. Verified SAFE (closed, with evidence)

- **No server-side SSRF** in any ingest path - `media/upload` and `enrich-one` operate on locally-present bytes only (`smackpress-api.php`:251, `gyss-api.php`:609-611).
- `image-ingest.php` is the **strongest file in the set** - extension allowlist, MIME re-derived from content, GD re-encode strips polyglot tails, GIF re-encoded, `upload-execution-guard.php` written into the upload dir, `is_uploaded_file/move_uploaded_file` gate.
- `smackpress-sanitize_html` is a proper DOM allowlist (strips `script/style/iframe/svg` with content, drops `on*/javascript:/data:`); **SVG sanitizer** blocks DOCTYPE/ENTITY + `LIBXML_NONET`; **skin install** is Ed25519-verified + zip-slip-guarded; **directory-feed SSRF** pinned with `CURLOPT_RESOLVE`.
- **RawTherapee/XPANO argument hygiene correct** (`argv` list, `shell=False`/QProcess args, `abspath`, no attacker sidecar) - the instruction layer is closed.
- `ImageMath.eval` is **not injectable** (fixed literal expressions, numeric-only `float()` coercion).

- **Shared-library store contained** - store_media ext-allowlist + contained_local_path commonpath jail (fixed in 053); content-addressed names can't clobber; cached post bodies are re-sanitised server-side on re-ingest, so the SQLite cache is not a trusted-execution bus.
- **No pickle / yaml.load on untrusted data**; eval/exec hits are build tooling only.
- **Path jails solid** across snap_home/snap_paths and the GYSS Rust resolve_in_root; **Zip Slip** in restore/skin-install mitigated; **no forgeable desktop auto-update-execute channel** exists.

5. Fixes applied in this pass

- **GYSS data-id XSS sink** - escHtml(p.id) at both renderSortGrid/renderGridGrid (commit on branch).
- store_media **ext-jail + allowlist** - carried from 053 (44ff1bd7).

All other items are staged below; the code-exec-carrying ones (CSP, credential vault, image safe-open, server-side config validation) are prioritised for the beta bar.

6. Shortcomings of the prior pass (053) - honest post-mortem

053 ("desktop-suite audit-fixes branch") was **competent but narrow, and narrow in the way that matters**: it was a **branch-DIFF review** - it audited the lines that changed, not the doors the changes (and the pre-existing suite) left open. Specifically it fell short on:

1. **Wrong altitude.** "Did this diff introduce a bug" is not "did we open the screen door during bug season." A diff review structurally cannot find a hole that already existed in code the diff didn't touch (the null CSP, the image-decoder surface, the base64 credentials were all invisible to it).
2. **Mistook an asking-side control for a close.** It treated the client-side GYSS wrong-site guard as *closing* the wrong-site CRITICAL. That guard is an assertion by the asking party; the enforcing close is server-side (mutual-auth A1), which 053 never examined.
3. **Never reached the RCE-richest surfaces:** image-decoder ingress (SNAP SLAPPER), config/profile poisoning (a CRITICAL found only this pass), credential-at-rest, the update/restore channel, or the CMS->desktop boundary at all.
4. **Scope was set by the diff, not by the threat model** - so its coverage was accidental, not deliberate.

None of 053's findings were wrong; the failure was one of *scope and altitude*. It answered "is this change safe" when the real duty-of-care question was "is this attack surface defended."

7. How we audit more carefully going forward

The fix is method, and this pass demonstrates it. Standing rules for every security pass touching the suite:

1. **Chokepoint coverage, not diff coverage.** Enumerate every boundary where untrusted data crosses (the 6 chokepoints) and check each - so coverage is deliberate, never accidental. A pass is not "done" because the changed lines are clean.
2. **Two-plus independent adversarial reviewers, never a single self-audit** (050). This pass used four; it caught the config-poisoning CRITICAL and the still-open CSP that the single narrow pass missed. Reviewers ground in live code and cross-check; a severity disagreement is itself a finding to resolve with Sean.
3. **Rate by FAIL-CLOSED enforcement at the boundary, not "today's caller is safe."** store_media was safe-via-caller and still hardened; that is the bar.
4. **Enforce at the ACTING boundary, not the asking one.** A client-side check is advisory; the close lives where the action happens (server rejects the write; desktop verifies the signature). Never file an asking-side control as a CRITICAL close.
5. **Evidence for every conclusion, safe or unsafe.** A closed item needs file:line proof as much as an open one.

6. **Never claim "done" on code presence or a headless build.** Code-exec-carrying controls (CSP, updater) require a build + live test; that gate is part of the audit, not after it (the GYSS CSP is explicitly held open here for exactly this reason).
7. **Non-code duty of care is part of the bar** (below) - hardening code is not the whole obligation.

The finite bar for beta (Nov 4), from the duty-of-care brief:

- The adversarial pass run and findings **CLOSED** for the code-execution-carrying chokepoints: 1 (image ingress), 3 (JS/IPC + CSP), 4 (update/restore), 5 (credentials), 6 (server parser).
- **mutual-auth A1 confirmed:** the server independently rejects an out-of-scope write (the true close on wrong-site).
- A **vulnerability-disclosure path** published (SECURITY.md, sibling to ETHICS.md).
- **Rollback tested off the happy path;** a **dependency-CVE watch** on the bundled C/Rust parsers (LibRaw, lcms, Qt, SQLite, Tauri); a **user-notification** channel for a critical issue.
- **MEMENTO MORI raises the floor:** it holds deceased photographers' archives - the most irreplaceable trust in the project, held for people who can no longer consent to a risk. Any tool touching those archives clears this bar *before* it touches them.

8. Open items -> priority + sign-off

Code, in-lane (Claude), reversible - do next:

1. snap_imgsafe.safe_open shared module + route the three untrusted decode paths through it (coordinate SNAP SLAPPER edits with Codex).
2. Server-side config validation at the connection sink: insecure_transport_reason + site_url==filename check (F1/F2).
3. GYSS download_to origin-pin + no-redirect + private-range block; job-object/timeout wrapper for RawTherapee/XPANO.
4. Server: RSS CURLOPT_RESOLVE fix; sanitise AI title/caption at storage.

Needs a build + live test (held open, not blind):

5. GYSS CSP (hash the importmap) + withGlobalTauri:false (rewrite the shim off the global) + exe-target write refusal + Hub roster-exe hash-verify -> tauri build + all-four-tabs test.

Three-way sign-off (Sean + Claude + Codex), architecture / duty-of-care:

6. Credential-at-rest: mandatory vault/keychain (owner UX decision).
7. mutual-auth A1 (server-side scope enforcement) + config signing (B1).
8. Backup-archive signing + verify-before-restore.
9. SECURITY.md, dependency-CVE watch, tested rollback, user-notification channel.

9. Closure checklist

- [x] Four-reviewer adversarial pass across all six chokepoints, grounded in live code
- [x] GYSS data-id XSS sink escaped; store_media ext-jail (053) confirmed
- [x] Server ingest confirmed free of SSRF; image-ingest/HTML/SVG/skin hardening confirmed
- [x] RawTherapee/XPANO instruction layer confirmed closed; parser layer = containment target
- [x] Prior-pass (053) shortcomings documented; go-forward method recorded (sections 6-7)

- [x] snap_imgsafe safe-open module + route untrusted decodes - tools/_shared/snap_imgsafe.py, used by 5 tools; SLAPPER decoders in bounded worker processes (056/057)
- [x] Profile site_url validation at the connection sink (CRITICAL F1) - snap_profiles: site_url must reduce to the filename it is stored under; discovery refuses insecure transport before the hub key goes out
- [x] Discovery node validation (F2) - 0.7.724D: snap_discovery.node_url_reason(); hub-supplied nodes must be a clean https origin (no user:pass@, no query/fragment, no private/loopback/link-local literal); rejected nodes are listed in hub_info["rejected_nodes"], never fixed up. Domain-allowlist policy still three-way (§10). Test: tools/_shared/tests/test_snap_discovery_node_urls.py
- [x] GYSS CSP + withGlobalTauri:false + exe-write refusal - in tauri.conf.json (hashed importmap CSP), lib.rs jail + refuse_executable, Hub roster-exe sha256 pin (tools/hub/main.py). Built into GYSS exe 2026-09-13.
Live all-four-tabs test: Sean to confirm (§10)
- [x] GYSS download_to origin-pin + no-redirect + private-range block - GYSS 0.7.11 (0.7.724D): host must equal the profile's site, redirects refused, private/loopback/link-local literals refused. Needs a GYSS rebuild to reach the exe
- [x] RawTherapee/XPANO containment - Windows Job Object (memory, timeout, descendants) per 056/057; restricted-token sandbox remains documented residual
- [x] Credential-at-rest vault - tools/_shared/snap_vault.py + SUYB secret_vault.py exist and are tested;
mandatory vs opt-in is the open three-way decision (§10)
- [x] Backup archive sign + verify-before-restore - **DROPPED 2026-09-18, owner decision**. Built that afternoon (HMAC over a per-member hash list, local key), then reverted the same day. Sean: "why are we responsible for knowing what was in a person's cloud storage? ... it's not our headache and we have taken on more than our share." The user owns their archive and their cloud; SUYB does not vouch for a file it did not keep. This was a three-way item and should not have been built unilaterally - noted. The two halves, so nobody re-litigates this: **we don't vouch for their archive (signing DROPPED - their file, their cloud, their job) / we don't wreck their disk opening it (058 B KEPT - our tool, running on their PC, is ours)**. Same file, opposite reasons
- [x] RSS DNS-rebind fix - CURLOPT_RESOLVE pin in core/fediverse.php, multisite-api.php, directory-api.php, cron-directory-feeds.php, smack-media-proxy.php
- [x] AI title/caption sanitise at storage - core/gyss-api.php (strip_tags + snap_sanitize_alt)
- [x] Suite-wide: strip/refuse Bearer on cross-host redirect - requests strips it cross-host by library behaviour (verified 2.34.2); every urllib caller that attaches a credential now uses a redirect-refusing opener (7 files). Test: tests/test_urllib_no_redirect_with_credentials.py
- [x] SECURITY.md - exists at repo root
- [] mutual-auth A1 server-side scope enforcement + config signing (B1) - **three-way decision, not started**
- [] Mandatory vault (owner UX decision) - **three-way decision**
- [] Dependency-CVE watch, tested rollback, user-notification channel - **process items, not code**
- [] MEMENTO MORI gate before any tool touches deceased archives - no tool does yet; gate to be built with the first such tool

10. Closure - second review + fixes, Claude, 2026-09-18

Sean asked for every unresolved audit to be dealt with. 053 and 054 had sat since Sep 2 with the report never updated, so most of §8 read as OPEN when the code had already moved. Every checklist line above was re-verified against dev at v0.7.723D before being ticked; the three that were genuinely still open in code (download_to pinning, backup signing, node-list validation) were fixed in 0.7.724D, plus the suite-wide urllib redirect hardening 053 F named.

Labels: every code item CLOSED. **Three items are decisions, not defects, and stay OPEN until the three of us sit down:**

1. **Mandatory vault.** Recommendation: mandatory for any profile that holds a posting-capable key, opt-in for read-only keys. Cost: one passphrase prompt per session on the affected tools.

2. **Server-side scope enforcement (A1) + signed config (B1).** Recommendation:

do A1 (server refuses a key used outside its declared scope - it is the backstop for every client-side check above) and defer B1 until there is a second signer.

3. **Domain allowlist for discovered nodes.** Recommendation: nodes must share the hub's registrable domain OR be on an explicit per-hub allowlist the owner edits. Today a hub can name any https host.

Owed live checks (Sean): GYSS all-four-tabs on the Sep 13 build (or the 0.7.11 rebuild); one watched SUYB backup -> restore round-trip on 0.7.44 (bounded unpack).

Delivery owed: GYSS 0.7.11 rebuild; SUYB 0.7.44 rebuild.